

Matplotlib 시각화 기초

숫자표로 안 보이던 것이 그림 한 장에 드러납니다

plot title / xlabel / legend bar hist scatter subplot

2026년 8월 25일 (화) · 25일차 · 58차시

교안 17_01 선 그래프와 그래프 완성 · 17_02 분포·범주·관계 시각화

국립금오공과대학교 컴퓨터공학부 박민호

교안	다룬 범위	상태
17_01 선 그래프와 그래프 완성	plot · 옵션 · 제목 · 축 · 범례 · 저장 · 실습 1~6	완주
17_02 분포·범주·관계 시각화	막대 · 히스토그램 · 산점도 · 리포트 · 실습 1~8	완주
18_01 Seaborn 시작	강사님 말씀 — "시본은 내일부터"	다음 시간

이 책을 읽는 법

뱃지 · 코드블록 · 이전 회차와의 연결

1. 분류 뱃지

모든 항목 이름 앞에는 그것이 **무엇인지**를 알려주는 색 뱃지가 붙습니다. 이름만 외우면 나중에 반드시 헛갈립니다.

뱃지	무엇	판별 신호	오늘의 예
함수	라이브러리가 제공하는 함수	앞에 모듈 이름이 옵니다	<code>plt.plot()</code> <code>plt.bar()</code>
키워드	함수에 넘기는 이름 붙은 인자	등호 왼쪽에 옵니다	<code>color=</code> <code>bins=</code> <code>alpha=</code>
속성	객체가 가진 값	점 뒤에 있고 괄호가 없습니다	<code>plt.rcParams</code>
개념	문법이 아닌 사고 틀	코드로 쓸 수 없습니다	4단계 흐름 · 그래프 선택 기준
패턴	자주 쓰는 코드 흐름	여러 요소의 조합입니다	<code>figure</code> → <code>plot</code> → 꾸미기 → <code>savefig</code> → <code>show</code>

개념

오늘은 거의 전부가 함수입니다. `plt.` 뒤에 붙는 이름들이 모두 `pyplot`이라는 모듈이 제공하는 함수이기 때문입니다. 예외가 하나 있는데 `plt.rcParams`입니다. **괄호가 없으니 속성**이고, 실제로는 설정값을 담은 딕셔너리입니다. 그래서 `plt.rcParams["font.family"] = "Malgun Gothic"`처럼 대괄호로 값을 넣습니다.

2. 코드블록 읽는 법

회색 배경에 왼쪽 파란 바가 붙은 상자는 **파이썬 코드**입니다. 상자 아래 가로줄 밑에 초록색 ▶로 시작하는 줄은 **그 코드를 실제로 실행했을 때 나온 출력**입니다. 이 책의 모든 출력은 실제 실행 결과를 옮긴 것입니다.

예시

```
import pandas as pd
```

```
df = pd.read_csv("Data/17_열처리.csv", encoding="utf-8")  
print(df.groupby("배치")["온도"].mean().round(2))
```

▶ 배치

▶ 로트A 858.42

▶ 로트B 859.81

▶ 로트C 860.42

▶ Name: 온도, dtype: float64

주의

오늘 배우는 것은 **그림을 그리는 코드**라 출력이 숫자가 아니라 그래프입니다. 그래서 이 책에서는 **그래프 대신 그 그래프가 무엇을 보여주는지 숫자로 함께 실습**합니다. 그림은 직접 그려서 확인하시고, 여기 실린 숫자와 맞는지 대조해 보십시오. **숫자와 그림이 서로를 검증**합니다.

3. 이전 회차와의 연결

전처리 단원이 끝나고 **시각화 단원**이 시작됩니다. 22일차부터 24일차까지 결측·이상치·중복을 숫자로 다뤘다면, 오늘부터는 **그 숫자를 그림으로** 바꿉니다.



연결

오늘 배우는 히스토그램·산점도는 **이미 배운 개념의 그림 버전**입니다. 히스토그램은 20일차의 `value_counts`와 `pd.cut`이 하던 일을 그림으로 하고, 산점도는 21일차의 `corr()`이 준 숫자 하나를 **점 44개로 펼쳐** 보여줍니다. 그래서 오늘 새로 외울 개념은 거의 없고, **이미 아는 것을 어떻게 보여줄지**만 배웁니다.

CONTENTS

목차

오늘 배우는 것을 한눈에

장	제목	무엇을 배우나
CH 0	오늘의 지도	왜 그림으로 보는가 · 그래프 그리는 4단계
STEP 1	첫 그래프와 한글 폰트	import · plot · show · 글자 깨짐 해결
STEP 2	선 그래프 그리기	x 생략 · color · linestyle · marker · 여러 선
STEP 3	그래프 완성하기	제목 · 축 이름 · 눈금 · 범례 · 크기 · 격자 · 저장
STEP 4	막대 그래프	bar · barh · 색 리스트로 강조 · 실습 1·2
STEP 5	히스토그램	hist · bins · 분포 읽기 · 실습 3·4
STEP 6	산점도	scatter · alpha · axhline · 실습 5·6
STEP 7	결측 처리와 종합 리포트	실습 7·8 — subplot으로 2×2 리포트
부록 A	치트시트	오늘 나온 모든 함수를 한 표에
부록 B	오류·함정 사전	노트북 코드 교정 · 자주 걸리는 함정
부록 C	실습 하이라이트와 다음 시간	핵심 결론 정리 · Seaborn 예고

개념

오늘의 한 문장. "사람의 눈은 숫자 비교에 약하고 모양 인식에 강하다." 62·63·64·66·69·73·78이라는 일곱 숫자를 보면 개별 값만 읽힙니다. 같은 값을 선으로 이으면 **오른쪽 위로 상승한다**는 사실이 한눈에 들어옵니다. 시각화는 장식이 아니라 **분석 도구**입니다.

오늘의 지도

왜 그림으로 보는가

0-1. 숫자표와 그래프의 차이

보는 방식	7일 온도	알 수 있는 것
숫자표	62 · 63 · 64 · 66 · 69 · 73 · 78	개별 값만
선 그래프	(같은 값을 선으로 이음)	오른쪽 위로 상승하는 추세

설비 적용

설비 한 대에서 1초마다 측정하면 하루에 8만 6천 개가 넘는 숫자가 쌓입니다. 이걸 표로 읽는 것은 불가능합니다. 그림으로 바꾸면 추세가 꺾이는 지점, 값이 튀는 순간, 두 설비의 차이가 한눈에 보입니다. 그래서 시각화는 분석의 마지막 단계가 아니라 분석 내내 쓰는 눈입니다.

0-2. matplotlib과 pyplot

개념 matplotlib · pyplot · plt 큰 도구 상자와 자주 쓰는 공구함

정의 matplotlib(맷플롯립)은 파이썬 그래프 도구 전체이고, pyplot(파이플롯)은 그 안에서 자주 쓰는 기능을 모은 공구함입니다. plt는 그 별명입니다.

형태 `import matplotlib.pyplot as plt`

```
import matplotlib.pyplot as plt
# matplotlib 안의 pyplot을 plt로 부름
```

왜 쓰나 plt는 전 세계 공통 약속입니다. 인터넷의 모든 예제가 이 이름을 씁니다. 임의로 바꾸면 남의 코드를 가져다 쓸 때마다 이름을 고쳐야 합니다.

응용 별명을 쓰는 이유는 코드를 짧고 읽기 쉽게 만들기 위해서입니다. 별명이 없으면 그래프 한 줄이 이렇게 길어집니다.

```
# 별명 없이
import matplotlib.pyplot
matplotlib.pyplot.plot([1, 2, 3], [10, 20, 15])
matplotlib.pyplot.show()

# 별명을 쓰면
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [10, 20, 15])
plt.show()
```

개념

pd(pandas), np(numpy), plt(pyplot), sns(seaborn) — 넷은 **바꾸지 않는 약속**입니다. 이 네 줄이 데이터 분석 코드의 표준 첫머리입니다.

자주 하는 실수

import matplotlib as plt로 쓰면 안 됩니다. **pyplot**까지 지정해야 합니다. matplotlib 자체에는 plot() 함수가 없어서 AttributeError가 납니다.

0-3. 그래프 그리는 4단계 흐름



어떤 그래프든 이 순서입니다. 그리고 **가장 중요한 규칙**은 show()가 맨 마지막에 딱 한 번이라는 점입니다. 그리기와 꾸미기를 전부 끝낸 뒤에 show()를 부릅니다.

주의

show()를 중간에 부르면 그때까지 그린 것만 화면에 나오고 그래프가 닫힙니다. 그 뒤에 붙인 title()이나 legend()는 새 빈 그래프에 적용되어 아무 데도 안 보입니다. 강사님이 수업 중에 이 실수를 직접 보여 주셨습니다. show()는 그래프 작업의 마침표입니다.

0-4. 오늘 다루는 데이터

파일	행 × 열	공정	어디에 쓰나
17_열처리.csv	44 × 6	열처리 소입로 — 로트 3개	실습 1~6
17_열처리_공정.csv	120 × 8	같은 공정, 주간·야간 라인	실습 7·8

설비 적용

열처리는 금속을 고온으로 가열했다가 식혀 강도와 경도를 바꾸는 공정입니다. **소입로**(燒入爐)는 그 가열로이고, 이 데이터에서 온도는 **859도 근처**에서 관리됩니다. **OP값**은 히터의 출력 비율(0~100)이고, **CP값**은 카본 포텐셜 — 노 안 분위기의 탄소 농도입니다. 로트A·B·C는 한 번에 넣고 처리하는 제품 묶음입니다.

오늘 데이터 첫인상

```
import pandas as pd
```

```
df = pd.read_csv("Data/17_열처리.csv", encoding="utf-8")
print(df.shape)
print(df.head(3))
print(df["배치"].value_counts().to_dict())
print(df["상태"].value_counts().to_dict())
```

```
▶ (44, 6)
▶
▶ 측정시각  배치  온도  OP값  CP값  상태
▶ 0   09:00   로트A   861.9   42.9   0.451   정상
▶ 1   09:10   로트A   858.4   97.6   0.446   정상
▶ 2   09:20   로트A   858.7   89.8   0.450   정상
▶
▶ {'로트A': 15, '로트B': 15, '로트C': 14}
▶ {'정상': 42, '승온': 2}
```

44행이라 **눈으로도 볼 수 있는 크기**입니다. 시각화를 처음 배울 때는 이런 작은 데이터가 좋습니다. 그림이 이상하게 나왔을 때 원본을 직접 확인할 수 있기 때문입니다.

주의

상태 컬럼에 **승온이 2건** 있습니다. 승온은 온도를 올리는 중이라는 뜻이니 **정상 가동 상태가 아닙니다**. 이 2건이 뒤에서 이상치로 잡히는데, 24일차에 배운 대로 **노이즈가 아니라 신호**입니다. 지우면 안 되는 값입니다.

STEP 1

첫 그래프와 한글 폰트

import · plot · show · 글자 깨짐 해결

그래프 한 장을 띄우는 데 필요한 줄은 세 줄입니다. 그리고 그 앞에 **한 줄을 더 붙여야** 한글이 안 깨집니다.

1-1. 세 줄이면 그래프가 뜬다

함수 `plt.plot()` · `plt.show()` 점을 선으로 잇고 화면에 띄우기

정의 `plot()`은 x와 y를 같은 순서끼리 짝지어 점을 만들고 **그 점들을 순서대로 선으로 잇습니다.** `show()`는 그린 것을 화면에 띄웁니다.

형태 `plt.plot(x, y) → plt.show()`

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [10, 20, 15])
plt.show()
```

▶ (1,10) (2,20) (3,15) 세 점을 이은 꺾은선이 뜹니다

왜 쓰나 이 줄을 만나야 그래프가 나타납니다. 없으면 아무 일도 안 일어납니다. 강사님이 짚으신 대로 그래프가 안 보일 때 **가장 먼저 확인할 것이 마지막 줄에 `show()`가 있는지입니다.**

응용 x 자리에 **글자를 넣어도 됩니다.** 그러면 그 글자가 가로축 눈금이 됩니다. 다만 이때 x는 순서만 가질 뿐 간격의 의미가 사라집니다.

```
plt.plot(["1월", "2월", "3월"], [10, 20, 15])
plt.show()
```

▶ 가로축에 1월·2월·3월이 글자 그대로 찍힙니다

개념

`plot(x, y)`에서 **x와 y의 개수가 반드시 같아야** 합니다. 다르면 `ValueError: x and y must have same first dimension`이 납니다. 이것이 그래프가 안 나올 때 두 번째로 확인할 항목입니다.

주의

주피터 노트북에서는 `show()` 없이도 마지막 줄의 그래프가 자동으로 표시되기도 합니다. 하지만 `.py` 파일로 실행하면 반드시 필요하고, 노트북에서도 붙이는 편이 안전합니다.

1-2. x를 생략하면 어떻게 되나

함수 `plt.plot(y)` `y`만 넣으면 `x`는 0부터 자동

정의 값 하나만 넣으면 그 값이 `y`가 되고 `x`는 0, 1, 2, ... 로 자동 부여됩니다.

형태 `plt.plot([62, 63, 65, 68, 72])`

```
plt.plot([62, 63, 65, 68, 72])
plt.show()
```

`x`축은 0, 1, 2, 3, 4 자동

▶ 가로축이 0부터 4까지 찍힌 상승 곡선

왜 쓰나 `x`가 그냥 순서일 때 편합니다. 측정 순서대로 늘어놓은 센서값처럼 `x`에 특별한 의미가 없으면 굳이 만들 필요가 없습니다.

응용 강사님은 여기서 **난수로 데이터를 만들어** 여러 번 실행해 보이셨습니다. 매번 모양이 달라지니 `plot`이 실제로 값을 따라 그린다는 것이 눈에 들어옵니다.

```
import random

ls = [random.randint(30, 100) for _ in range(5)]
print(ls)
plt.plot(ls)
plt.show()
```

▶ [57, 91, 34, 78, 62] ← 실행할 때마다 달라짐

개념

왜 0부터인가. 파이썬은 순서를 0부터 세기 때문입니다. 리스트의 첫 원소가 `ls[0]`인 것과 같은 규칙입니다. 1부터 시작하게 하려면 `x`를 직접 만들어 넣어야 합니다.

주의

`range(1, 6)`을 `x`로 넘기면 1부터 시작합니다. 다만 `plot(range(1,6), y)`처럼 **`range` 객체를 그대로 넣어도** 동작합니다. 리스트로 바꿀 필요가 없습니다.

1-3. 한글이 깨지는 이유와 해결

제목이나 축 이름에 한글을 넣으면 **네모 상자로 나옵니다**. 강사님도 수업 첫머리에 이것부터 잡고 시작하셨습니다.

속성 `plt.rcParams` matplotlib 전역 설정을 담은 딕셔너리

정의 rc는 **runtime configuration**의 줄임말입니다. 폰트·색·크기 같은 기본 설정이 여기 들어 있고, 대괄호로 값을 바꿉니다.

형태 `plt.rcParams["font.family"] = "Malgun Gothic"`

```
import matplotlib.pyplot as plt

# 앞으로 모든 그래프 코드 맨 위에 붙일 준비물

plt.rcParams["font.family"] = "Malgun Gothic"    # 맥은 "AppleGothic"
plt.rcParams["axes.unicode_minus"] = False
```

왜 쓰나 코드 오류가 아니라 글꼴 설정 문제입니다. matplotlib의 기본 글꼴에는 영어와 숫자 모양만 들어 있어서, 한글 자리에 **자리만 잡고 네모를 그립니다**. 컴퓨터가 고장 난 것이 아니라 그릴 글자 모양을 못 찾은 것뿐입니다.

응용 두 번째 줄도 중요합니다. `axes.unicode_minus = False`는 **음수 부호가 깨지는 것을 막습니다**. 한글 폰트로 바꾸면 유니코드 마이너스 기호가 네모로 나오는 일이 생기는데, 이 줄이 그것을 일반 하이픈으로 대체합니다.

```
# 운영체제별 글꼴 이름
# 윈도우   : "Malgun Gothic"
# 맥       : "AppleGothic"
# 리눅스·코랩 : "NanumGothic" (설치 필요)

# 현재 적용된 폰트 확인
print(plt.rcParams["font.family"])
# ['Malgun Gothic']
```

개념

이 두 줄은 **파일 맨 위에 딱 한 번만** 쓰면 됩니다. 그래프마다 반복할 필요가 없습니다. 노트북이라면 첫 셀에 두고 커널을 켤 때 한 번 실행하면 그 세션 내내 유지됩니다.

자주 하는 실수

폰트 이름이 **정확히 맞아야** 합니다. 오타가 나거나 그 폰트가 없으면 경고가 뜨고 기본 글꼴로 돌아가 여전히 깨집니다. 경고 메시지에 `findfont: Font family ... not found`가 보이면 이름을 다시 확인하십시오.

1-4. 그래프가 안 보일 때 점검 순서

증상	원인	점검할 곳
아무것도 안 나옴	<code>show()</code> 누락	마지막 줄
길이 오류	x와 y의 개수가 다름	두 리스트의 len 비교
한글이 네모	글꼴 설정 누락	파일 맨 위 rcParams
꾸민 게 안 붙음	<code>show()</code> 를 중간에 부름	show가 맨 마지막인지

개념

교안의 한 줄이 좋습니다. "오류는 혼내는 게 아니라 어디가 문제인지 알려 주는 것." 시각화 코드는 오류 메시지가 친절한 편입니다. 길이가 안 맞으면 그렇게 말해 주고, 폰트가 없으면 그렇게 말해 줍니다. 메시지를 읽는 습관만 있으면 대부분 5초 안에 고칩니다.

STEP 2

선 그래프 그리기

color · linestyle · marker · 여러 선 겹치기

기본 선이 나왔으니 이제 **모양을 정합니다**. 옵션은 쉼표로 계속 이어 붙이면 됩니다.

2-1. 선 색 지정하기

키워드 color= 선 색을 정하기

정의 plot()에 쉼표로 이어 붙이는 옵션입니다. 색 이름이나 16진수 코드를 넣습니다.

형태 plt.plot(x, y, color="red")

```
plt.plot([1,2,3,4,5], [62,63,65,68,72], color="red")
plt.show()
```

▶ 빨간색 상승 선 그래프

왜 쓰나 색을 의미 있게 쓰면 위험한 설비를 한눈에 표시할 수 있습니다. 교안의 표현대로 빨강은 위험, 파랑은 안정입니다. 색은 장식이 아니라 **정보를 담는 자리**입니다.

응용 색 이름 말고 **16진수 코드**로도 지정할 수 있습니다. #RRGGBB 형식이고 앞 두 자리가 빨강, 가운데가 초록, 뒤가 파랑의 세기입니다.

```
# 색 이름 — 쉽지만 한정적
plt.plot(x, y, color="red")

# 16진수 — 자유롭게 지정
plt.plot(x, y, color="#FF5733")
```

개념

색상 코드는 **외우지 말고 color picker에서 골라 복사**하십시오. 보고서 그래프 색을 회사 색으로 통일하면 훨씬 전문적으로 보입니다.

주의

자주 쓰는 색 이름은 red · blue · green · orange · purple · gray · black입니다. 이 정도만 알아도 대부분 해결되고, 정확한 색이 필요할 때만 16진수를 씁니다.

2-2. 선 스타일과 마커

옵션	값	모양	언제
linestyle	'-' (기본)	실선	일반적인 추세
linestyle	'--'	점선	보조선 · 목표선 · 비교선
marker	'o'	동그라미	측정 지점을 표시하고 싶을 때
marker	's'	네모	두 번째 계열 구분

ipynb/32_matplotlib_line_graph.ipynb

```
x = [1, 2, 3, 4, 5]
y = [855, 857, 860, 862, 865]
```

옵션은 쉼표로 계속 이어 붙인다

```
plt.plot(x, y, color="red", linestyle="--", marker="o")
plt.show()
```

▶ 빨간 점선에 동그라미 마커가 찍힌 상승 그래프

개념

마커를 붙이면 실제 측정 지점이 어디인지 보입니다. 마커 없이 선만 있으면 그 사이 값도 측정된 것처럼 보이는데, 사실은 점과 점 사이를 직선으로 이었을 뿐입니다. 측정 간격이 성긴 데이터일수록 마커가 중요합니다. 열처리 데이터처럼 10분 간격 측정이라면 반드시 붙이는 편이 정직합니다.

2-3. 여러 선 겹쳐 그리기

패턴 plot 여러 번 + label + legend 설비끼리 비교하기

정의 plot()을 여러 번 부르면 show() 전까지 그린 모든 선이 한 그래프에 겹칩니다. 각 선에 label을 달고 마지막에 legend()를 부르면 이름표가 붙습니다.

형태 plt.plot(x, y, label="이름") → plt.legend()

```
import numpy as np

x = np.arange(10)
y1 = x * 2
y2 = x * 3

plt.plot(x, y1, label="1호기")
plt.plot(x, y2, label="2호기")
plt.legend()
plt.show()
```

▶ 두 직선이 겹쳐 그려지고 오른쪽에 1호기·2호기 범례가 뜹니다

왜 쓰나 설비끼리 비교하는 가장 기본적인 방법입니다. 같은 축 위에 놓아야 어느 쪽이 높은지, 언제 역전되는지가 보입니다.

응용 색을 지정하지 않으면 **matplotlib이 알아서 다르게 골라 줍니다**. 기본 색 순서가 정해져 있어 파랑 → 주황 → 초록 순으로 붙습니다. 세 개 이하면 그냥 두어도 구분됩니다.

```
# 반복문으로 여러 설비를 한 번에
lots = ["로트A", "로트B", "로트C"]
colors = ["red", "blue", "green"]

for lot, c in zip(lots, colors):
    sub = df[df["배치"] == lot]
    plt.plot(sub["측정시각"], sub["온도"], label=lot, color=c)

plt.legend()
plt.show()
```

▶ 로트 세 개의 온도 추세가 각각 다른 색으로 겹쳐 그려집니다

개념

응용 코드의 `zip(lots, colors)`가 핵심입니다. 두 리스트를 **짜지어 동시에 돌립니다**. 설비 이름과 색을 따로 관리하다가 한 번에 꺼내 쓰는 방식이라, 설비가 늘어나도 리스트만 고치면 됩니다.

자주 하는 실수

`label`만 달고 `legend()`를 안 부르면 **범례가 안 나옵니다**. 둘은 반드시 짝입니다. `label`은 이름을 적어 두는 것이고, `legend`는 그 이름을 보여 주는 것입니다.

2-4. 데이터는 어디서 오는가

교안의 표현대로 "그래프 그리기는 데이터 준비 + plot 명령이고, 데이터 준비가 절반"입니다. 데이터 출처는 두 가지입니다.

데이터 출처 두 가지

```
import os
```

```
# 1. 직접 만든 리스트
```

```
plt.plot([1, 2, 3], [62, 63, 65])
```

```
# 2. CSV에서 불러온 데이터프레임 열
```

```
file_path = os.path.join("../", "Data", "17_열처리_공정.csv")
```

```
df = pd.read_csv(file_path, encoding="utf-8")
```

```
plt.plot(df["측정시각"], df["소입로온도1존"])
```

```
plt.show()
```

▶ 둘 다 plot에 그대로 들어갑니다

주의할 점	왜	확인 방법
숫자인지 확인	글자가 섞이면 축이 이상해짐	<code>df.dtypes</code>
결측치 미리 처리	NaN 자리에서 선이 끊김	<code>df.isna().sum()</code>
날짜는 정렬	순서가 뒤죽박죽이면 선이 왔다갔다	<code>sort_values("측정시각")</code>

연결

세 항목 모두 **22일차부터 24일차까지 배운 것**입니다. 시각화가 전처리 다음에 오는 이유가 여기 있습니다. **깨끗하지 않은 데이터로 그림을 그리면 그림이 거짓말을 합니다.** 결측치 있는 자리에서 선이 뚝 끊기고, 이상치 하나 때문에 나머지가 전부 바닥에 깔려 보입니다.

자주 하는 실수

강사님 노트북에 이런 줄이 있습니다. `plt.plot(df["측정시각"], df["result"])`. `result`는 정상·이상이라는 **글자 컬럼**입니다. 실행하면 오류 없이 그려지는데, matplotlib이 글자를 카테고리 처리해 **y축이 정상·이상 두 칸짜리가 되고 선이 그 사이를 오갑니다.** 선 그래프로서는 의미가 없습니다. 이 경우 세는 그래프(막대)나 `result`를 숫자로 바꾼 뒤 그리는 편이 맞습니다. 자세한 내용은 부록 B에 실었습니다.

STEP 3

그래프 완성하기

제목 · 축 · 눈금 · 범례 · 크기 · 격자 · 저장

선만 그린 그래프는 **미완성**입니다. 그린 사람도 며칠 뒤면 무슨 그래프인지 모릅니다. 여기서 배우는 것은 전부 **읽는 사람**을 위한 장치입니다.

3-1. 제목과 축 이름

함수 `plt.title()` · `plt.xlabel()` · `plt.ylabel()` 무슨 그래프인지 알려주기

정의 `title`은 그래프 위쪽 가운데, `xlabel`은 가로축 이름, `ylabel`은 세로축 이름입니다. **x**는 가로, **y**는 세로, **label**은 이름표로 외우면 됩니다.

형태 `plt.title("제목")` · `plt.xlabel("가로")` · `plt.ylabel("세로")`

```
plt.plot([1,2,3,4,5], [858, 859, 860, 862, 865])
plt.title("로트C 시각별 소입로 온도")
plt.xlabel("측정 순서")
plt.ylabel("온도(도씨)")
plt.show()
```

▶ 제목과 두 축 이름이 붙은 완성형 선 그래프

왜 쓰나 단위가 없으면 숫자의 의미가 사라집니다. 세로축에 860이 찍혀 있어도 그것이 도씨인지 화씨인지 압력인지 알 수 없습니다. 축 이름에는 **반드시 단위까지** 적습니다.

응용 제목에도 좋고 나쁨이 있습니다. 교안이 든 예가 명확합니다.

```
# x 나쁜 제목 — 무엇을 보는지 알 수 없다
plt.title("그래프")
plt.title("데이터")

# ✓ 좋은 제목 — 설비명·값·기준이 드러난다
plt.title("로트C 시각별 소입로 온도 (관리 기준 859도)")
```

개념

좋은 제목은 **설비명 · 무슨 값 · 어떤 기준**이 드러나게 씁니다. 그래프가 회의 자료로 넘어가면 **제목만 읽고 넘기는 사람**이 대부분입니다. 제목 한 줄이 그래프 전체의 요약이 됩니다.

주의

제목·축 이름은 반드시 `show()` 앞에 씁니다. `show()` 뒤에 쓰면 새 빈 그래프에 붙어 아무 데도 안 보입니다.

3-2. 축 눈금 바꾸기

함수 `plt.xticks()` 가로축 눈금 글자를 직접 교체

정의 데이터는 그대로 두고 보여 주는 방식만 바꿉니다. 위치 리스트와 글자 리스트를 순서대로 짝지어 넘깁니다.

형태 `plt.xticks(위치리스트, 글자리스트)`

```
plt.plot([1,2,3,4,5], [62,63,65,68,72])
plt.xticks([1,2,3,4,5], ["월", "화", "수", "목", "금"])
plt.show()
```

▶ 가로축 1~5 자리에 월·화·수·목·금이 표시됩니다

왜 쓰나 x가 숫자여야 계산이 편한데 보여 줄 때는 글자가 읽기 좋은 경우가 많습니다. 데이터를 건드리지 않고 표시만 바꾸는 것이 이 함수의 역할입니다.

응용 항목 이름이 길어 겹칠 때는 `rotation`으로 비스듬히 눕힙니다.

```
plt.bar(["1학기", "2학기", "3학기", "4학기"], [64, 70, 77, 68])
plt.xticks(rotation=45)
plt.show()
```

▶ 가로축 이름이 45도 기울어져 겹치지 않습니다

개념

위치 리스트와 글자 리스트의 길이가 같아야 합니다. 다르면 오류가 나거나 일부만 표시됩니다. 순서대로 짝지어진다는 점도 기억하십시오.

자주 하는 실수

`xticks`로 바꾼 것은 표시일 뿐 데이터가 아닙니다. 그래프에 월·화·수가 보여도 안에 든 값은 여전히 1·2·3입니다. 다른 계산에 쓸 때 헛갈리지 마십시오.

3-3. 범례 위치와 그래프 크기

키워드 `legend(loc=)` · `figure(figsize=)` 범례 위치와 도화지 크기

정의 `loc`는 범례를 놓을 모서리, `figsize`는 그래프 전체 크기를 **인치 단위 (가로, 세로)**로 지정합니다.

형태 `plt.legend(loc="upper left")` · `plt.figure(figsize=(10, 4))`

```
# figure는 반드시 plot보다 먼저
plt.figure(figsize=(10, 4))
plt.plot(x, y, label="로트A")
plt.legend(loc="upper left")
plt.grid(True, alpha=0.3)
plt.show()
```

▶ 가로로 긴 그래프에 왼쪽 위 범례와 흐린 격자가 붙습니다

왜 쓰나 `loc`는 데이터가 비어 있는 쪽으로 범례를 보내는 것이 요령입니다. 상승 그래프면 왼쪽 위가 비고, 하강 그래프면 오른쪽 위가 비니다. 범례가 선을 가리면 그래프를 망칩니다.

응용 `figsize`는 `plot`보다 반드시 먼저 **와야 합니다**. 도화지를 먼저 펴고 그 위에 그리는 순서이기 때문입니다. 나중에 부르면 새 도화지가 만들어져 앞에 그린 것이 사라집니다.

```
# loc 값 네 가지
# "upper left"   왼쪽 위
# "upper right"  오른쪽 위
# "lower left"   왼쪽 아래
# "lower right"  오른쪽 아래

# 격자는 alpha로 흐리게 — 주인공이 아니므로
plt.grid(True, alpha=0.3)
```

개념

`grid()`의 `alpha`는 **투명도**입니다. 0이 완전 투명, 1이 완전 불투명입니다. 0.3쯤이면 값 읽기는 도와주면서 데이터를 가리지 않습니다. **격자는 주인공이 아니라 보조선**입니다.

주의

`figsize`가 너무 크면 오히려 안 좋습니다. 강사님도 짚으셨듯 **너무 넓거나 높으면 한 화면에 안 들어옵니다**. 보고서에 넣을 거면 (10, 4)나 (8, 5) 정도가 무난합니다.

3-4. 파일로 저장하기

함수 `plt.savefig()` 그래프를 이미지 파일로 저장

정의 그린 그래프를 이미지로 내보냅니다. 반드시 `show()`보다 먼저 써야 합니다.

형태 `plt.savefig("파일명.png", dpi=150, bbox_inches="tight")`

```
plt.plot([1,2,3], [62,65,72])
plt.title("로트C 온도 변화")
plt.savefig("로트C_온도.png") # show 보다 먼저
plt.show()
```

▶ 작업 폴더에 로트C_온도.png 가 생깁니다

왜 쓰나 `show()`가 그래프를 닫아 버리기 때문입니다. `show()` 뒤에 `savefig()`를 쓰면 이미 비워진 도화지를 저장해 빈 이미지 파일이 생깁니다.

응용 두 옵션을 함께 쓰면 보고서에 바로 붙일 품질이 나옵니다. `dpi`는 해상도이고 `bbox_inches="tight"`는 여백을 잘라 냅니다.

```
plt.savefig("리포트.png", dpi=150, bbox_inches="tight")
```

```
# 확장자별 특징
# .png 글자가 또렷 — 리포트용 표준
# .jpg 용량 작음 — 사진형 이미지에 적합
# .pdf 인쇄용 — 확대해도 안 깨짐
```

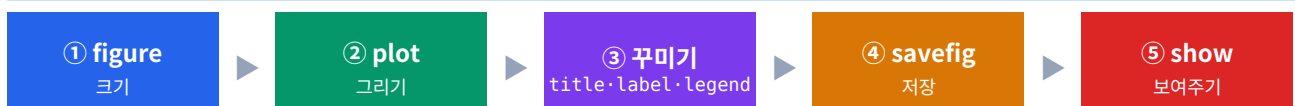
개념

리포트용 그래프는 **png**를 씁니다. 글자가 또렷하게 남기 때문입니다. jpg는 글자 주변이 뭉개져 축 이름이 흐려집니다.

자주 하는 실수

저장 파일 이름에 **한글을 쓰면 환경에 따라 깨질 수 있습니다**. 안전하게 가려면 영문·숫자·밑줄만 쓰십시오.

3-5. 코드 순서가 결과를 바꾼다



위치	반드시 지킬 것	안 지키면
맨 앞	<code>figure</code>	앞에 그린 것이 사라짐
중간	<code>plot</code> → 꾸미기	순서 자유
show 직전	<code>savefig</code>	빈 이미지가 저장됨
맨 뒤	<code>show</code>	꾸민 게 안 붙음

개념

figure 맨 앞, show 맨 뒤, savefig는 show 바로 앞. 이 세 문장만 외우면 순서 문제는 다 해결됩니다. 그 사이의 꾸미기 명령들은 순서가 자유롭습니다. 제목을 먼저 달든 범례를 먼저 달든 결과가 같습니다.

3-6. 읽을 수 있는 그래프의 조건

점검 항목	확인 내용	빠지면
제목	무슨 그래프인지 한눈에 알 수 있는가	며칠 뒤 본인도 모름
축 이름	두 축의 의미와 단위가 적혀 있는가	숫자의 뜻을 알 수 없음
범례	선이 여럿일 때 구분이 되는가	어느 선이 어느 설비인지 모름
크기·격자	값을 읽기 편한가	대략만 보이고 정확히 못 읽음

설비 적용

교안의 문장이 정확합니다. **"화려함보다 기본을 빠뜨리지 않는 것이 중요하다."** 같은 데이터로 그려도 **제목·축·범례가 빠지면 신뢰도가 갑니다.** 현장 회의에 제목 없는 그래프를 들고 가면 설명하는 데 시간을 다 쓰게 됩니다. 반대로 잘 단 라벨 하나가 긴 설명을 대신합니다.

STEP 4

막대 그래프

bar · barh · 색 리스트로 강조

여기서부터 교안 17_02입니다. 선 그래프가 **이어지는 흐름**을 보여준다면, 막대는 **떨어진 항목의 비교**를 보여줍니다.

4-1. 선과 막대, 언제 무엇을

구분		
보는 것	이어지는 변화	떨어진 항목 비교
가로축	시간처럼 연속된 축	서로 관계없는 이름
예	요일이 지나며 온도가 변함	로트A·B·C의 평균 온도
질문	어떻게 변했나	어느 쪽이 큰가

개념

가르는 기준은 **가로축이 이어지는가**입니다. 시각·날짜·순서처럼 **사이**를 이어도 의미가 있으면 **선**, 로트A와 로트B처럼 **사이에 아무것도 없으면 막대**입니다. 로트A와 로트B 사이의 중간값이라는 것은 존재하지 않으므로 선으로 이으면 거짓말이 됩니다.

4-2. 실습 1 — 설비별 평균 온도 막대

함수 `plt.bar()` 범주별 값을 막대 높이로

정의 첫 자리에 **항목 이름**, 둘째 자리에 **값**을 넣습니다. `plot`과 거의 같은 방식입니다.

형태 `plt.bar(항목리스트, 값리스트, color=..., width=...)`

```
df_1 = pd.read_csv("Data/17_열처리.csv", encoding="utf-8")
```

```
# groupby로 설비별 평균 온도를 구해 막대로  
mean_temp = df_1.groupby("배치")["온도"].mean()  
print(mean_temp.round(2))
```

```
plt.bar(mean_temp.index, mean_temp.values)  
plt.title("로트별 평균 온도")  
plt.ylabel("온도(도씨)")  
plt.show()
```

```
▶ 배치  
▶ 로트A      858.42  
▶ 로트B      859.81  
▶ 로트C      860.42  
▶ Name: 온도, dtype: float64
```

왜 쓰나 20일차에 배운 groupby가 그대로 그림이 됩니다. 집계 결과의 index가 항목 이름, values가 막대 높이입니다. 판다스와 matplotlib이 이렇게 이어집니다.

응용 색 리스트를 넣으면 막대마다 다른 색이 됩니다. 가장 뜨거운 로트만 빨강으로 강조하는 식입니다.

```
# 가장 뜨거운 로트만 빨강으로  
colors = ["blue", "blue", "red"]  
plt.bar(mean_temp.index, mean_temp.values, color=colors, width=0.5)  
plt.show()
```

```
# 가로 막대 — 이름이 길거나 항목이 많을 때  
plt.barh(mean_temp.index, mean_temp.values)  
plt.xlabel("온도(도씨)") # 값이 가로로 늘어나므로 xlabel  
plt.show()
```

개념

barh의 h는 **horizontal**입니다. 사용법은 bar와 같지만 **값이 가로로 늘어나므로 단위를 xlabel에 적어야 합니다. 항목 이름이 길거나 순위표를 만들 때 보기 좋습니다.**

자주 하는 실수

세 로트의 평균 온도가 858.42 · 859.81 · 860.42로 **2도 차이**입니다. 그런데 막대 그래프가 **세로축을 858부터 시작하면 로트C가 로트A의 몇 배처럼 보입니다.** 이것이 다음 절에서 다룰 함정입니다.

4-3. 막대 그래프가 사람을 속이는 법

막대는 쉬워 보이지만 **잘못 그리면 사람을 속이는 그래프**입니다. 교안이 한 장을 따로 할애했습니다.

항목	올바른 방법	안 지키면
세로축 시작	반드시 0부터	2도 차이가 두 배로 과장됨
항목 순서	값 크기 순으로 정렬	어디가 문제인지 안 보임
색	강조할 것만 다른 색	전부 알록달록하면 초점이 흐려짐

주의

오늘 데이터가 정확히 그 예입니다. **로트별 평균 온도가 858.42 · 859.81 · 860.42로 차이가 2도밖에 안 됩니다.** 그런데 세로축을 858에서 861까지로 잡으면 로트A 막대는 거의 안 보이고 로트C만 우뚝 섭니다. **0.23%의 차이가 몇 배 차이처럼 보이는 것입니다.** 열처리 공정에서 2도는 의미 있는 차이일 수 있지만, **그림이 그 차이를 과장하면 안 됩니다.**

직접 확인해 볼 것

축 시작점에 따라 인상이 달라진다

```
plt.bar(mean_temp.index, mean_temp.values)
```

```
plt.ylim(0, 900)          # 0부터 — 세 막대가 거의 같아 보임
```

```
plt.show()
```

```
plt.bar(mean_temp.index, mean_temp.values)
```

```
plt.ylim(858, 861)       # 858부터 — 차이가 크게 과장됨
```

```
plt.show()
```

▶ 같은 데이터인데 두 그래프의 인상이 완전히 다릅니다

정답은 **둘 다 상황에 따라 맞다**입니다. 0부터 그리면 정직하지만 2도 차이가 안 보이고, 858부터 그리면 차이는 보이지만 과장됩니다. 어느 쪽을 쓰든 **축 눈금을 명확히 표시하고, 차이가 몇 도인지 숫자로 함께 적는 것이** 정직한 방법입니다.

4-4. 실습 2 — 개수와 평균을 막대로

Practice/17_02_example.ipynb — 실습 2

```
# 상태 열의 개수를 세어 막대로

state_count = df_2["상태"].value_counts()
print(state_count)

plt.bar(state_count.index, state_count.values)
plt.title("상태별 측정 건수")
plt.show()

# 배치로 그룹을 나눠 평균 OP값을 막대로
mean_op = df_2.groupby("배치")["OP값"].mean()
print(mean_op.round(2))
```

```
▶ 상태
▶ 정상 42
▶ 승온 2
▶ Name: count, dtype: int64
▶
▶ 배치
▶ 로트A 81.91
▶ 로트B 75.84
▶ 로트C 69.62
▶ Name: OP값, dtype: float64
```

개념

여기서 **결정적인 관찰**이 나옵니다. 평균 온도는 로트A 858.42 → 로트C 860.42로 **올라가는데**, 평균 OP값은 로트A 81.91 → 로트C 69.62로 **내려갑니다**. 정확히 반대 방향입니다. 두 막대 그래프를 나란히 놓으면 이 대칭이 한눈에 들어옵니다. 왜 그런지는 STEP 6의 산점도에서 밝혀집니다.

STEP 5

히스토그램

hist · bins · 분포 읽기

막대와 생김새는 비슷한데 **목적**이 **완전히 다른** 그래프입니다. 이 둘을 혼동하는 것이 입문자가 가장 많이 하는 실수입니다.

5-1. 막대와 히스토그램의 차이

구분		
보는 것	정해진 항목별 크기 비교	숫자가 퍼진 분포 확인
가로축	서로 떨어진 범주 (로트A·B·C)	연속된 구간 (855~860도)
막대 사이	띄어져 있음	붙어 있음
넣는 것	항목과 값 두 개	숫자 목록 하나
질문	어느 쪽이 큰가	어디에 몰려 있나

개념

막대 사이가 붙어 있는지 떨어져 있는지가 눈으로 가르는 신호입니다. 히스토그램은 가로축이 **끊김 없이 이어진 구간**이라 막대가 서로 맞닿아 있습니다. 막대 그래프는 로트A와 로트B 사이에 아무것도 없으므로 떨어져 있습니다. **그림만 봐도 어느 쪽인지 알 수 있어야** 합니다.

5-2. 실습 3 — 히스토그램 그리기와 bins

함수 `plt.hist()` 숫자 목록 하나로 분포 그리기

정의 숫자 목록 하나만 넣으면 값의 범위를 같은 너비 구간으로 쪼개고, 각 구간에 몇 개가 들어가는지 세어 막대 높이로 그립니다.

형태 `plt.hist(숫자목록, bins=구간수)`

```
df_3 = pd.read_csv("Data/17_열처리.csv", encoding="utf-8")
```

```
# 온도 열을 hist로 그려 자동 구간 분포 확인
```

```
plt.hist(df_3["온도"])
```

```
plt.xlabel("온도(도씨)")
```

```
plt.ylabel("측정 건수")
```

```
plt.title("소입로 온도 분포")
```

```
plt.show()
```

▶ 859~861도 근처에 막대가 높게 솟고 양쪽으로 낮아지는 종 모양

왜 쓰나 20일차에 배운 `pd.cut()`과 `value_counts()`가 하던 일을 **한 줄로, 그림으로** 합니다. 구간 경계를 직접 정하지 않아도 알아서 나눠 줍니다.

응용 `bins`가 히스토그램에서 **가장 중요한 옵션**입니다. 구간을 몇 개로 나눌지 정하는데, **분포 모양이 이 값에 따라 달라집니다**.

```
# bins를 바꿔 가며 같은 데이터를 보기
```

```
for b in [3, 10, 30]:
```

```
    plt.hist(df_3["온도"], bins=b)
```

```
    plt.title(f"bins={b}")
```

```
    plt.show()
```

▶ `bins=3` : 세 덩어리 — 봉우리 위치만 대략 보임

▶ `bins=10` : 적절 — 중심과 퍼짐이 함께 보임

▶ `bins=30` : 막대가 잘게 쪼개져 들쭉날쭉

개념

정답인 bins는 없습니다. 너무 작으면 859도 근처인지 861도 근처인지 세부가 사라지고, 너무 크면 우연한 요철이 진짜 패턴처럼 보입니다. 교안이 준 출발점은 **데이터 개수의 제곱근**입니다. 44건이면 약 6~7이 시작점입니다.

주의

여러 bins로 그려 보고 일관되게 나타나는 특징만 신뢰하십시오. `bins`를 3에서 30까지 바꿔도 봉우리가 계속 859도 근처에 있다면 그것은 진짜입니다. `bins`를 바꿀 때마다 사라지는 요철은 무시해도 됩니다.

5-3. 분포를 읽는 세 단계

① 중심
어디 모였나



② 퍼짐
얼마나 흩어졌나



③ 동떨어진 값
끝에 뭐가 있나

단계	무엇을 보나	오늘 데이터	현장 의미
① 중심	막대가 가장 높은 구간	859~860도	정상 작동 범위
② 퍼짐	좁게 모였나 넓게 흩어졌나	표준편차 2.105도	설비 안정성
③ 끝	동떨어진 막대가 있나	852도 · 865.6도	이상 후보 설비

개념

그릴 줄 아는 것과 읽을 줄 아는 것은 다른 일입니다. 히스토그램은 세 가지만 보면 됩니다. 어디에 모였나(중심), 얼마나 퍼졌나(퍼짐), 끝에 뭐가 있나(이상 후보). 이 세 질문을 **매번 체크리스트처럼** 따라가면 어떤 분포든 읽을 수 있습니다.

5-4. 실습 4 — 분포를 통계와 함께 해석

그림만 보고 끝내면 절반입니다. **평균과 표준편차를 함께 구해 그림과 대조**하는 것이 실습 4의 목적입니다.

Practice/17_02_example.ipynb — 실습 4

```
# 평균과 표준편차를 구해 분포의 중심·퍼짐과 비교
mean_t = df_4["온도"].mean()
std_t = df_4["온도"].std()
print("평균:", round(mean_t, 3), "| 표준편차:", round(std_t, 3))
```

```
# 조건 필터링으로 이상 설비 찾기
high = df_4[df_4["온도"] >= 862]
print("862도 이상 측정 건수:", len(high))
print(high[["측정시각", "배치", "온도", "상태"]])
```

```
▶ 평균: 859.532 | 표준편차: 2.105
▶ 862도 이상 측정 건수: 2
▶
▶ 측정시각 배치 온도 상태
▶ 25 10:40 로트B 862.9 정상
▶ 32 09:30 로트C 865.6 정상
```

평균 859.532는 히스토그램의 봉우리 위치와 맞습니다. 표준편차 2.105는 봉우리의 폭과 맞습니다. 그림과 숫자가 서로를 검증한 것입니다.

설비 적용

862도 이상이 2건인데 둘 다 상태가 정상으로 기록되어 있습니다. 865.6도는 평균보다 표준편차 세 배만큼 높은 값입니다. 24일차에 배운 IQR로 계산하면 상한이 863.075이니 **865.6은 명백한 이상치**입니다. 그런데 기록상 정상입니다. 기록이 잘못됐거나, 순간적인 오버슛이거나, 센서 문제일 수 있고 — 어느 쪽인지는 현장이 답합니다.

히스토그램의 양 끝을 숫자로 확인

24일차에 배운 IQR로 같은 데이터를 확인

```
q1, q3 = df_4["온도"].quantile(0.25), df_4["온도"].quantile(0.75)
iqr = q3 - q1
lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
print(f"경계 {lo:.3f} ~ {hi:.3f}")
```

```
m = (df_4["온도"] < lo) | (df_4["온도"] > hi)
print(df_4[m][["측정시각", "배치", "온도", "상태"]])
```

▶ 경계 856.075 ~ 863.075

▶

▶ 측정시각 배치 온도 상태

▶ 7 10:10 로트A 852.1 승온

▶ 12 11:00 로트A 854.3 승온

▶ 32 09:30 로트C 865.6 정상

연결

이상치 3건 중 2건이 상태 승온입니다. 24일차에 배운 **신호성 이상치**의 완벽한 사례입니다. 승온 중이라 온도가 낮은 것이고, **지우면 안 되는 값**입니다. 나머지 하나인 865.6도만 노이즈인지 신호인지 따져 보면 됩니다. **IQR이 후보를 짚어 주고 상태 컬럼이 절반을 걸러 준 것**입니다.

STEP 6

산점도

scatter · alpha · axhline · 두 값의 관계

선 그래프가 **값 하나의 변화**를, 히스토그램이 **값 하나의 분포**를 봤다면, 산점도는 **두 값의 관계**를 봅니다.

6-1. 산점도의 원리

함수 plt.scatter() 두 값을 가로·세로에 두고 점 찍기

정의 x와 y를 모두 지정해 **점만 찍고 잊지 않습니다**. 선 그래프와의 결정적 차이입니다.

형태 plt.scatter(x, y, color=..., s=..., alpha=...)

```
df_5 = pd.read_csv("Data/17_열처리.csv", encoding="utf-8")

# 온도와 OP값을 산점도로 찍어 관계 확인
plt.scatter(df_5["온도"], df_5["OP값"])
plt.xlabel("온도(도씨)")
plt.ylabel("OP값(%)")
plt.title("온도와 제어 출력의 관계")
plt.show()

print("상관계수:", round(df_5["온도"].corr(df_5["OP값"]), 3))
```

▶ 상관계수: -0.783

▶

▶ 점들이 왼쪽 위에서 오른쪽 아래로 내려가는 모양

왜 쓰나 왜 잊지 않는가. 점 44개는 서로 다른 44번의 측정이지 하나의 흐름이 아닙니다. 이으면 없는 순서를 만들어 내는 셈입니다. **관계는 점들이 만드는 전체 모양**에서 읽습니다.

응용 점이 많아 겹칠 때는 alpha로 반투명하게 만듭니다. 그러면 겹친 곳이 진하게 보여 밀도까지 드러납니다.

```
# 점 색·크기·투명도
plt.scatter(df_5["온도"], df_5["OP값"],
            color="steelblue", s=60, alpha=0.6)
plt.show()

# s는 점 크기, alpha는 0(투명)~1(불투명)
```

개념

s는 **size**입니다. 점이 작아 안 보이면 키우고, 너무 뭉치면 줄입니다. 44건 정도면 기본값으로도 충분하지만 수천 건이면 s를 줄이고 alpha를 낮춰야 모양이 보입니다.

주의

축 배치에는 관행이 있습니다. **원인이 될 변수를 x, 결과가 될 변수를 y**에 놓습니다. 다만 산점도 자체는 인과를 말해 주지 않으므로, 이것은 읽는 사람을 위한 배려일 뿐입니다.

6-2. 산점도가 보여주는 세 패턴

점들의 모양	관계	오늘 데이터
오른쪽 위로 올라감	양의 관계 — 함께 커짐	—
오른쪽 아래로 내려감	음의 관계 — 하나 크면 하나 작음	온도-OP값 -0.783
둥글게 흩어짐	관계 없음	온도-CP값 0.022

연결

21일차에 배운 `corr()`이 준 숫자 하나가 **여기서 점 44개로 펼쳐집니다**. -0.783이라는 숫자만 보면 "강한 음의 관계"라는 말밖에 못 하지만, 산점도를 보면 **어떤 모양으로 내려가는지, 벗어난 점이 어디 있는지까지** 보입니다. 숫자는 요약이고 그림은 원본입니다.

6-3. 실습 5 — 온도와 OP값의 관계

상관계수 **-0.783**은 강한 음의 관계입니다. 그런데 이것이 **이상 신호일까요, 정상일까요**. 오늘 문서에서 가장 중요한 질문입니다.

양 끝값에서 답이 나옵니다

양 끝을 직접 확인해 보기

```
print(df_5.nlargest(2, "온도")[[ "측정시각", "배치", "온도", "OP값", "상태"]])  
print(df_5.nsmallest(2, "온도")[[ "측정시각", "배치", "온도", "OP값", "상태"]])
```

```
▶ 측정시각 배치 온도 OP값 상태  
▶ 32 09:30 로트C 865.6 7.7 정상  
▶ 25 10:40 로트B 862.9 28.5 정상  
▶  
▶ 측정시각 배치 온도 OP값 상태  
▶ 7 10:10 로트A 852.1 92.0 승온  
▶ 12 11:00 로트A 854.3 100.0 승온
```

설비 적용

온도가 가장 높은 865.6도일 때 OP값이 7.7%입니다. 온도가 가장 낮은 852.1도일 때 OP값이 92.0%이고, 854.3도일 때는 100%입니다.

OP값은 히터의 출력 비율입니다. 즉 온도가 낮으면 히터를 세게 돌리고, 온도가 높으면 끕니다. 이것은 이상 신호가 아니라 온도 제어가 정상 작동한다는 증거입니다.

그리고 OP값 100%인 두 건이 상태 승온으로 기록되어 있습니다. 승온 = 온도를 올리는 중 = 히터 최대 출력.

데이터가 물리적으로 완벽히 일관됩니다.

21일차에 배운 상관 ≠ 인과를 여기서 다시 만납니다. 온도와 OP값은 강하게 연동되지만, 온도가 OP값을 낮춘 것도 아니고 OP값이 온도를 올린 것도 아닙니다. 둘 사이에 제어기라는 장치가 있어 서로를 조절하는 것입니다. 데이터만 봐서는 이 구조를 알 수 없고, 공정을 알아야 읽힙니다.

6-4. 기준선 그리기

함수 plt.axhline() · plt.axvline() 가로·세로 기준선 긋기

정의 ax는 축(axis), h는 수평(horizontal), v는 수직(vertical)입니다. 임계값을 그래프 위에 표시할 때 씁니다.

형태 plt.axhline(y=값, color=..., linestyle="--")

```
plt.scatter(df_5["온도"], df_5["OP값"], alpha=0.6)

# 관리 상한선을 빨간 점선으로
plt.axvline(x=862, color="red", linestyle="--", label="온도 상한")
plt.axhline(y=90, color="orange", linestyle="--", label="OP 경고")

plt.legend()
plt.show()
```

▶ 산점도 위에 세로 빨간 점선과 가로 주황 점선이 그어집니다

왜 쓰나 점이 기준을 넘었는지 눈으로 바로 판정할 수 있습니다. 숫자를 하나씩 대조할 필요 없이 선 오른쪽·위쪽에 있는 점만 세면 됩니다.

응용 기준선을 상태별 색과 함께 쓰면 훨씬 강력해집니다. 상태를 색으로 매핑하는 방법입니다.

```
# 상태를 색으로 매핑
color_map = {"정상": "blue", "승온": "red"}
colors = df_5["상태"].map(color_map)

plt.scatter(df_5["온도"], df_5["OP값"], c=colors, alpha=0.7)
plt.axhline(y=90, color="orange", linestyle="--")
plt.show()
```

▶ 승온 2건이 빨간 점으로, 왼쪽 위 구석(저온·고출력)에 몰려 나타납니다

개념

.map(딕셔너리)는 Series의 각 값을 **딕셔너리 규칙에 따라 바꿔** 줍니다. 상태 문자열을 색 이름으로 바꾸는 데 딱 맞습니다. 그리고 scatter에서는 색 인자가 color가 아니라 **c**입니다.

주의

linestyle="--"로 **점선**을 쓰는 것이 관행입니다. 기준선은 데이터가 아니라 **참고선**이므로 실선으로 그리면 데이터와 헷갈립니다.

6-5. 실습 6 — 관계가 없는 경우도 결론이다

Practice/17_02_example.ipynb — 실습 6

```
# 실습 5와 같은 방식으로 온도와 CP값의 산점도
for col in ["OP값", "CP값"]:
    r = df_6["온도"].corr(df_6[col])
    print(f"온도 ~ {col} : {r:.3f}")
```

▶ 온도 ~ OP값 : -0.783

▶ 온도 ~ CP값 : 0.022

0.022는 사실상 0입니다. 산점도를 그리면 점들이 방향 없이 둥글게 흩어져 있습니다. 21일차의 강도 기준으로는 거의 무상관입니다.

설비 적용

이것도 **정상입니다**. CP값은 카본 포텐셜 — 노 안 분위기의 탄소 농도이고, 온도와는 **별도로 제어되는 값**입니다. 따로 조절하니 서로 연동될 이유가 없습니다. **관계가 없다는 것도 하나의 결론**입니다. "온도만 보면 CP값을 짐작할 수 없다"는 사실을 알게 된 것이고, 이는 **CP값을 따로 감시해야 한다**는 뜻입니다.

두 산점도를 나란히 놓으면 오늘 배운 것이 요약됩니다. **같은 온도 축에 대해 하나는 뚜렷한 내리막이고 하나는 구름 모양입니다.** 숫자로는 -0.783과 0.022라는 두 값이지만, 그림으로 보면 **왜 그 숫자가 나왔는지**까지 보입니다.

STEP 7

결측 처리와 종합 리포트

subplot · 2×2 대시보드

오늘의 마지막 단계입니다. 22·23일차의 결측 처리 + 25일차의 시각화를 하나로 묶어 한 장짜리 공정 리포트를 만듭니다.

7-1. 실습 7 — 결측이 있는 데이터를 시각화하기 전에

실습 7부터는 데이터가 바뀝니다. 17_열처리_공정.csv는 120행 8열이고 결측이 25개 들어 있습니다.

Practice/17_02_example.ipynb — 실습 7

```
df_7 = pd.read_csv("Data/17_열처리_공정.csv", encoding="utf-8")
print(df_7.shape)
print(df_7.isna().sum())
```

- ▶ (120, 8)
- ▶
- ▶ 측정시각 0
- ▶ 라인 0
- ▶ 소임로온도1존 0
- ▶ 소임로온도2존 8
- ▶ CP값 6
- ▶ OP값 6
- ▶ 세정기온도 5
- ▶ result 0
- ▶ dtype: int64

자주 하는 실수

결측이 있는 열을 그냥 그리면 그림에 구멍이 생깁니다. plt.plot()은 NaN 자리를 선이 끊긴 채로 그리고, plt.hist()는 NaN을 조용히 빼고 셉니다. 오류가 나지 않으므로 그림이 잘못됐다는 것을 알아채기 어렵습니다. 그래서 시각화 전에 isna().sum()으로 결측을 먼저 확인하는 습관이 필요합니다. 22일차에 배운 그 메시드가 이제 그리기 전 점검 도구가 된 것입니다.

메서드 .fillna() 결측을 지정한 값으로 채우기 — 시각화 준비

정의 23일차에 배운 그대로입니다. 다만 오늘은 그래프를 그리기 위한 전처리로 씁니다.

형태 df[열].fillna(df[열].mean())

```
# 숫자 열의 결측을 각 열의 평균으로 대체
num_cols = ["소입로온도2존", "CP값", "OP값", "세정기온도"]

for c in num_cols:
    df_7[c] = df_7[c].fillna(df_7[c].mean())

print("남은 결측:", df_7.isna().sum().sum())
```

▶ 남은 결측: 0

왜 쓰나 그리기 전에 채워 두면 **선이 끊기지 않고 막대 높이도 정확**해집니다. 25개 중 하나라도 남으면 그 열의 그래프는 실제와 다른 그림이 됩니다.

응용 대체가 데이터를 얼마나 바꿨는지 **반드시 확인**해야 합니다. 평균 대체는 값을 중앙에 몰아넣으므로 **퍼짐이 줄어듭니다**.

```
# 대체 전후 표준편차 비교
raw = pd.read_csv("Data/17_열처리_공정.csv", encoding="utf-8")

before = raw["소입로온도2존"].std()
after = df_7["소입로온도2존"].std()
print(f"대체 전 {before:.3f} → 대체 후 {after:.3f}")
```

▶ 대체 전 0.350 → 대체 후 0.338

개념

표준편차가 **0.350에서 0.338로 줄었습니다**. 8건을 모두 평균값으로 채웠으니 그만큼 퍼짐이 좁아진 것입니다. 8/120이라 변화가 작지만, **결측 비율이 크면 이 왜곡도 커집니다**. 23일차에 배운 "평균 대체는 분산을 줄인다"가 숫자로 확인된 것입니다.

주의

히스토그램을 그릴 때는 평균 대체가 특히 위험합니다. **대체한 평균값 자리에 없던 봉우리가 솟기** 때문입니다. 분포를 볼 목적이라면 `dropna()`가 더 정직할 때가 많습니다.

7-2. 라인별로 갈라 보기

21일차 groupby가 여기서 다시 쓰입니다

```
# 라인과 result를 교차표로
print(pd.crosstab(df_7["라인"], df_7["result"]))

# 라인별 평균 온도
print(df_7.groupby("라인")["소입로온도1존"].mean().round(3))
```

```
▶ result    이상 정상
▶ 라인
▶ 야간      8      52
▶ 주간      3      57
▶
▶ 라인
▶ 야간    860.043
▶ 주간    859.968
▶ Name: 소입로온도1존, dtype: float64
```

설비 적용

야간 이상 8건 대 주간 이상 3건 — 야간이 두 배 넘게 많습니다. 그런데 평균 온도는 860.043 대 859.968로 거의 같습니다. 평균만 보면 두 라인은 똑같아 보이는데 결과는 다릅니다. **평균이 감추고 있는 무언가가 있다**는 뜻이고, 그것을 찾으려면 **분포를 그려 봐야** 합니다. 21일차의 "평균이 같아도 퍼짐이 다르면 다른 공정이다"가 바로 이 상황입니다.

7-3. 실습 8 — subplot으로 한 장 리포트 만들기

함수 plt.subplot() 한 그림 안에 여러 그래프 배치

정의 그림판을 행 × 열 격자로 나누고 몇 번째 칸에 그릴지 정합니다. 번호는 1부터 시작해 왼쪽에서 오른쪽, 위에서 아래 순서입니다.

형태 plt.subplot(행, 열, 번호)

```
plt.figure(figsize=(12, 8))

# 1번 칸 — 온도 추이 (선)
plt.subplot(2, 2, 1)
plt.plot(df_8["소입로온도1존"], color="steelblue")
plt.title("온도 추이")
```

```

# 2번 칸 — 온도 분포 (히스토그램)
plt.subplot(2, 2, 2)
plt.hist(df_8["소입로온도1존"], bins=12, color="seagreen")
plt.title("온도 분포")

# 3번 칸 — 라인별 이상 건수 (막대)
plt.subplot(2, 2, 3)
cnt = df_8[df_8["result"] == "이상"]["라인"].value_counts()
plt.bar(cnt.index, cnt.values, color=["tomato", "silver"])
plt.title("라인별 이상 건수")

# 4번 칸 — 온도와 OP값 (산점도)
plt.subplot(2, 2, 4)
plt.scatter(df_8["소입로온도1존"], df_8["OP값"], alpha=0.5)
plt.title("온도 vs OP값")

plt.tight_layout()
plt.savefig("공정리포트.png", dpi=150)
plt.show()

```

- ▶ 2행 2열로 네 개의 그래프가 배치된 한 장짜리 리포트
- ▶ 3번 칸: 야간 8건(빨강) · 주간 3건(회색)

왜 쓰나 보고서에 그림 네 장을 따로 붙이는 것보다 **한 장에 모으면 서로 연결해서 읽힙니다**. 추이·분포·비교·관계를 한눈에 보는 것이 **공정 대시보드의 기본형**입니다.

응용 `plt.subplot(2,2,1)` 대신 `plt.subplots(2,2)`(끝에 s)를 쓰면 축 객체를 한꺼번에 받아 다루기 편합니다. 다만 **함수 이름이 한 글자 차이로 헷갈리기 쉽습니다**.

```

# subplots — 축 객체 방식
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

axes[0, 0].plot(df_8["소입로온도1존"])
axes[0, 1].hist(df_8["소입로온도1존"], bins=12)
axes[1, 0].bar(cnt.index, cnt.values)
axes[1, 1].scatter(df_8["소입로온도1존"], df_8["OP값"])

plt.tight_layout()
plt.show()

```

- ▶ 같은 2×2 배치가 나옵니다

개념

`plt.subplot()`(단수)은 칸을 하나씩 골라 가며 그리고, `plt.subplots()`(복수)는 격자를 한 번에 만들어 축 배열을 돌려줍니다. 오늘 교안은 단수형을 씁니다. 축 배열은 `axes[행][열]`로 접근하는데 행·열 번호가 0부터 시작합니다. `subplot`의 번호는 1부터인데 `axes`의 인덱스는 0부터라 두 방식을 섞으면 헷갈립니다.

주의

`plt.tight_layout()`을 빼먹으면 제목과 축 이름이 서로 겹칩니다. 그래프를 여러 개 넣을 때는 거의 항상 필요하다고 생각하십시오.

함수 `plt.tight_layout()` 겹친 글자 자동 정리

정의 그래프들 사이 여백을 자동 계산해 제목·축 이름·눈금이 겹치지 않게 재배치합니다.

형태 `plt.tight_layout()`

```
# 반드시 그린 뒤, show/savefig 앞에서 호출
plt.subplot(2, 2, 1); plt.plot([1, 2, 3]); plt.title("첫 번째")
plt.subplot(2, 2, 2); plt.plot([3, 2, 1]); plt.title("두 번째")

plt.tight_layout()
plt.show()
```

왜 쓰나 2×2 배치에서는 위쪽 그래프의 x축 이름과 아래쪽 그래프의 제목이 거의 항상 겹칩니다. 이 한 줄이면 해결됩니다.

응용 `figsize`와 함께 조절하면 더 확실합니다. 칸이 4개면 가로 12 세로 8 정도가 편안합니다.

```
# 여백을 직접 조절하고 싶을 때
plt.subplots_adjust(hspace=0.35, wspace=0.25)
# hspace: 위아래 간격, wspace: 좌우 간격
```

주의

호출 순서가 중요합니다. 모든 그래프를 그린 뒤에, `plt.show()`나 `plt.savefig()` 앞에서 불러야 합니다. 먼저 부르면 아무 효과가 없습니다.

7-4. 리포트를 읽는 순서



칸	그래프	묻는 질문	오늘 데이터의 답
1	선	시간에 따라 어떻게 변했나	859~861도 사이에서 진동
2	히스토그램	값이 어디에 몰렸나	860도 중심의 종 모양
3	막대	어느 쪽이 문제인가	야간 8 : 주간 3
4	산점도	두 값이 연동되나	온도-OP값 음의 관계

설비 적용

네 칸을 순서대로 읽으면 **하나의 이야기**가 됩니다. "온도는 대체로 안정적이고(①), 정규분포에 가까우며(②), 그런데 이상은 야간에 몰려 있고(③), 온도와 출력은 제어기로 묶여 있다(④)." 여기서 나오는 다음 질문은 **"야간에만 뭐가 다른가"**입니다. 야간 근무 인원, 외기 온도, 설비 예열 상태 — 데이터 밖의 요인을 찾아야 할 차례입니다. **좋은 리포트는 답이 아니라 다음 질문을 줍니다.**

그리고 이 네 칸을 만드는 데 쓴 것은 오늘 배운 `plot · hist · bar · scatter` 네 함수가 전부입니다. 나머지는 21~24 일차에 배운 **pandas**입니다. 새 도구 네 개로 이만큼 할 수 있다는 것이 **오늘의 성과**입니다.

치트시트

오늘 배운 것을 한 장으로

A-1. 그래프 종류 고르기

묻고 싶은 것	그래프	함수	최소 코드
시간에 따라 어떻게 변했나	선	<code>plt.plot()</code>	<code>plt.plot(y)</code>
항목별로 어느 쪽이 큰가	막대	<code>plt.bar()</code>	<code>plt.bar(x, y)</code>
값이 어디에 몰려 있나	히스토그램	<code>plt.hist()</code>	<code>plt.hist(y, bins=10)</code>
두 값이 연동되나	산점도	<code>plt.scatter()</code>	<code>plt.scatter(x, y)</code>
이름이 길다 / 항목이 많다	가로 막대	<code>plt.barh()</code>	<code>plt.barh(x, y)</code>

A-2. 꾸미기 함수

함수	하는 일	자주 쓰는 형태
<code>plt.title()</code>	그래프 제목	<code>plt.title("온도 추이", fontsize=14)</code>
<code>plt.xlabel()</code>	가로축 이름	<code>plt.xlabel("측정시각")</code>
<code>plt.ylabel()</code>	세로축 이름	<code>plt.ylabel("온도(도씨)")</code>
<code>plt.legend()</code>	범례 표시	<code>plt.legend(loc="upper right")</code>
<code>plt.xticks()</code>	가로 눈금 조정	<code>plt.xticks(rotation=45)</code>
<code>plt.figure()</code>	그림 크기 지정	<code>plt.figure(figsize=(12, 6))</code>
<code>plt.grid()</code>	격자 표시	<code>plt.grid(True, alpha=0.3)</code>
<code>plt.ylim()</code>	세로축 범위 고정	<code>plt.ylim(0, 100)</code>
<code>plt.axhline()</code>	가로 기준선	<code>plt.axhline(y=862, color="red")</code>
<code>plt.axvline()</code>	세로 기준선	<code>plt.axvline(x=10, linestyle="--")</code>
<code>plt.savefig()</code>	파일로 저장	<code>plt.savefig("a.png", dpi=150)</code>
<code>plt.subplot()</code>	여러 칸에 배치	<code>plt.subplot(2, 2, 1)</code>
<code>plt.tight_layout()</code>	겹친 글자 정리	<code>plt.tight_layout()</code>

함수	하는 일	자주 쓰는 형태
<code>plt.show()</code>	화면에 띄우기	<code>plt.show()</code>

A-3. 스타일 옵션

옵션	값	의미
<code>color=</code>	"red" · "steelblue" · "#2563EB"	선·막대·점의 색
<code>linestyle=</code>	"-" 실선 · "--" 점선 · ":" 점 · "-." 일점쇄선	선 모양
<code>marker=</code>	"o" 원 · "s" 사각 · "^" 삼각 · "*" 별	데이터 점 표시
<code>linewidth=</code>	1 ~ 3	선 굵기
<code>alpha=</code>	0 (투명) ~ 1 (불투명)	투명도 — 겹칠 때
<code>s=</code>	20 ~ 100	산점도 점 크기
<code>bins=</code>	데이터 수의 제곱근이 출발점	히스토그램 구간 수
<code>label=</code>	"1존" — <code>legend()</code> 와 짝	범례에 표시할 이름

A-4. 한글 폰트 설정 — 매번 첫 줄에

복사해서 노트북 첫 셀에 붙여 두십시오

```
import pandas as pd
import matplotlib.pyplot as plt

# 한글 깨짐 방지 (Windows)
plt.rcParams["font.family"] = "Malgun Gothic"

# 마이너스 기호 깨짐 방지
plt.rcParams["axes.unicode_minus"] = False
```

주의

macOS는 "AppleGothic", 리눅스는 "NanumGothic"을 씁니다. `axes.unicode_minus`를 False로 안 하면 음수 눈금의 마이너스가 네모로 나옵니다. 온도 차이나 상관계수처럼 음수가 나오는 그래프에서 반드시 필요합니다.

A-5. 그래프 하나를 완성하는 표준 순서

이 순서를 외우면 대부분의 그래프를 만들 수 있습니다

① 크기 지정 — 그리기 전에

```
plt.figure(figsize=(12, 6))
```

② 데이터 그리기 — 여러 번 겹칠 수 있음

```
plt.plot(df["소입로온도1존"], label="1존", color="steelblue")
```

```
plt.plot(df["소입로온도2존"], label="2존", color="tomato", linestyle="--")
```

③ 설명 붙이기

```
plt.title("소입로 존별 온도 추이", fontsize=14)
```

```
plt.xlabel("측정 순번")
```

```
plt.ylabel("온도(도씨)")
```

```
plt.legend(loc="upper right")
```

```
plt.grid(True, alpha=0.3)
```

④ 저장 → 화면 — savefig가 show보다 먼저

```
plt.savefig("온도추이.png", dpi=150, bbox_inches="tight")
```

```
plt.show()
```

오류·함정 사전

X 이렇게 하면 → ✓ 바로잡기

B-1. 문자열 컬럼을 x축에 넣기

무슨 일이 일어나

X 측정시각이 "09:00" 같은 문자열인데 그대로 x축에 넣음

```
plt.plot(df["측정시각"], df["result"])
```

▶ 눈금이 겹쳐 읽을 수 없고, result가 문자열이라 세로축도 의미 없음

바로잡기

✓ 세로축에는 숫자를, 눈금이 많으면 기울이거나 숨아냄

```
plt.figure(figsize=(14, 5))
```

```
plt.plot(df["측정시각"], df["소입로온도1존"])
```

```
plt.xticks(rotation=45)
```

또는 일정 간격으로만 눈금 표시

```
plt.xticks(range(0, len(df), 10), df["측정시각"][::10], rotation=45)
```

개념

matplotlib은 문자열도 x축에 받아 줍니다. 오류가 안 나기 때문에 더 위험합니다. 문자열 120개를 넣으면 눈금 120개를 다 찍으려 해서 글자가 서로 뭉개집니다. 그리고 result 같은 범주형(정상/이상)을 세로축에 넣으면 두 줄 사이를 왔다 갔다 하는 무의미한 선이 됩니다. 범주형 비교에는 막대를 쓰십시오.

B-2. 자주 만나는 오류 여덟 가지

X 이렇게 하면	무슨 일이	✓ 바로잡기
<code>import matplotlib</code>	<code>plt.plot</code> 없음 — <code>AttributeError</code>	<code>import matplotlib.pyplot as plt</code>
<code>plt.plot(df)</code>	모든 숫자 열이 한꺼번에 그려짐	<code>plt.plot(df["온도"])</code>
<code>plt.legend()</code>	빈 범례 — <code>label=</code> 을 안 줌	<code>plt.plot(y, label="1존")</code>
<code>plt.figure()</code>	plot 뒤에 쓰면 빈 그림만 새로 생김	plot 앞에 배치

X 이렇게 하면	무슨 일이	✓ 바로잡기
<code>plt.show()</code>	show 뒤에 savefig 하면 빈 파일 저장	savefig → show 순서
한글 제목	글자가 □□□ 로 깨짐	<code>plt.rcParams["font.family"]</code>
음수 눈금	마이너스가 네모로 나옴	<code>axes.unicode_minus = False</code>
<code>plt.subplot(2,2,0)</code>	번호가 1부터 — 0은 오류	<code>plt.subplot(2, 2, 1)</code>

B-3. 그림은 맞는데 결론이 틀리는 경우

함정	왜 위험한가	대응
막대 세로축 자르기	0.5% 차이가 두 배처럼 보임	<code>plt.ylim(0, ...)</code> 으로 0부터
bins를 하나만 보기	우연한 요철을 패턴으로 착각	bins를 3·10·30으로 바꿔 확인
결측을 안 채우고 그리기	선이 끊기거나 조용히 빠짐	<code>isna().sum()</code> 먼저 확인
평균 대체 후 히스토그램	없던 봉우리가 평균 자리에 생김	분포를 볼 땐 <code>dropna()</code> 고려
상관 산점도로 인과 주장	온도-OP값 -0.783 은 제어기 때문	공정 지식으로 해석
점을 선으로 잇기	없는 순서를 만들어 냄	관계는 scatter, 흐름은 plot

개념

B-2는 오류가 나는 실수라 금방 알아챱니다. B-3은 그림이 멀쩡하게 그려지는데 결론만 틀리는 실수라 훨씬 위험합니다. 특히 막대 세로축 자르기는 발표 자료에서 흔하게 볼 수 있고, 악의 없이도 자주 일어납니다. 내가 그린 그래프가 남을 오해시키지 않는지 항상 한 번 더 보십시오.

실습 하이라이트와 다음 시간

오늘의 결론 · 예고

C-1. 실습 1~8 한눈에

실습	데이터	한 일	핵심 결과
1	17_열처리.csv	배치별 평균 온도 막대	로트A 858.42 → 로트C 860.42
2	//	배치별 평균 OP값 막대	로트A 81.91 → 로트C 69.62
3	//	온도 히스토그램	bins에 따라 모양이 달라짐
4	//	평균·표준편차와 대조	평균 859.532 · 표준편차 2.105
5	//	온도-OP값 산점도	상관 -0.783 — 제어기의 흔적
6	//	온도-CP값 산점도	상관 0.022 — 무관도 결론
7	17_열처리_공정.csv	결측 확인 후 평균 대체	결측 25개 → 0
8	//	2×2 종합 리포트	야간 이상 8 : 주간 3

C-2. 오늘 데이터가 말해 준 것

실습을 흠어진 여덟 개가 아니라 **하나의 조사 과정**으로 다시 읽어 보겠습니다.

설비 적용

- ① **실습 1·2에서 이상한 것을 발견했습니다.** 평균 온도는 로트A→C로 올라가는데(858.42 → 860.42) 평균 OP 값은 내려갑니다(81.91 → 69.62). **정확히 반대 방향**입니다.
- ② **실습 5에서 원인을 찾았습니다.** 산점도를 그리니 상관 -0.783의 뚜렷한 내리막이 보였고, 양 끝을 확인하니 **865.6도일 때 OP 7.7%, 852.1도일 때 OP 92%**였습니다. 히터 출력이 온도를 따라 반대로 움직이는 것 — **피드백 제어**입니다.
- ③ **실습 4·6에서 검증했습니다.** 862도 이상 2건은 IQR 상한 863.075를 기준으로 하나만 진짜 이상치였고, OP값 100%인 두 건은 모두 **상태 승온**으로 기록되어 있어 물리적으로 일관됐습니다. CP값은 온도와 무관(0.022)해 별도 감시 대상임을 확인했습니다.
- ④ **실습 7·8에서 새 질문을 얻었습니다.** 평균 온도는 야간 860.043 대 주간 859.968로 거의 같은데 이상은 **야간 8 대 주간 3**입니다. 평균이 감추고 있는 무언가가 있습니다.

여기까지가 **탐색적 데이터 분석(EDA)의 전형적인 한 사이클**입니다. 그림을 보고 → 이상한 점을 찾고 → 숫자로 확인하고 → 새 질문을 얻습니다. 오늘 배운 네 함수가 이 사이클을 **한 바퀴 돌 수 있게 해 준 것**입니다.

C-3. 21~25일차 다섯 회차의 연결



회차	배운 것	오늘 어디서 다시 쓰였나
21일차	<code>groupby().mean() · corr()</code>	실습 1·2 막대 · 실습 5·6 산점도
22일차	<code>isna().sum()</code>	실습 7 — 그리기 전 결측 점검
23일차	<code>fillna()</code> · 평균 대체의 부작용	실습 7 — 표준편차 0.350 → 0.338
24일차	<code>quantile()</code> · IQR 경계	실습 4 — 히스토그램 양 끝 검증
25일차	<code>plot · bar · hist · scatter</code>	네 가지를 subplot으로 묶어 리포트

연결

오늘 새로 배운 matplotlib 함수는 **네 개**뿐입니다. 나머지는 전부 21~24일차에 배운 pandas입니다. **시각화는 새로운 분석이 아니라, 이미 하던 분석을 보여 주는 방법**입니다. 그래서 pandas를 모르면 그래프도 못 그리고, 그래프를 못 그리면 pandas 결과를 남에게 설득하지 못합니다.

C-4. 다음 시간 예고

오늘 마지막에 강사님이 예고하신 대로, 다음 시간부터는 **Seaborn**으로 넘어갑니다.

구분		
성격	기본 도구 — 직접 다 지정	matplotlib 위에 얹은 편의 도구
색 · 스타일	하나하나 지정	기본값이 이미 보기 좋음
그룹별 색 나누기	<code>.map()</code> 으로 직접 만들	<code>hue=</code> 한 줄
DataFrame 연동	열을 직접 꺼내 넘김	<code>data=df</code> 로 통째로
왜 먼저 배웠나	—	기초를 알아야 seaborn을 고칠 수 있음

개념

Seaborn을 쓰더라도 **제목·축·범례·저장은 결국 plt. 함수로 조정합니다. Seaborn은 matplotlib을 대체하는 것이 아니라 감싸는 것입니다. 오늘 배운 것이 다음 시간에 그대로 쓰입니다. 그리고 시계열 데이터를 다루는 내용도 이어질 예정입니다.**

차이를 미리 한 번만 보고 가겠습니다. 오늘 실습 5에서 **상태별 색을 칠하려고 .map()**으로 딕셔너리를 만들었던 부분이 다음 시간에 어떻게 바뀌는지입니다.

같은 그림, 다른 코드

오늘 — matplotlib 방식

```
color_map = {"정상": "blue", "승온": "red"}
colors = df["상태"].map(color_map)
plt.scatter(df["온도"], df["OP값"], c=colors, alpha=0.7)
plt.show()
```

다음 시간 — seaborn 방식

```
import seaborn as sns
sns.scatterplot(data=df, x="온도", y="OP값", hue="상태")
plt.show()
```

▶ 두 코드의 결과 그래프는 거의 같습니다

▶ 다만 seaborn 쪽은 범례까지 자동으로 붙습니다

`hue=` 하나가 딕셔너리 만들기 + 색 매핑 + 범례 붙이기 세 가지를 대신합니다. 편해 보이지만, **자동으로 정해진 색이 마음에 안 들 때** 고칠 수 있으려면 오늘 배운 `color=`와 `legend()`를 알아야 합니다. 그래서 기초를 먼저 한 것입니다.

오늘의 성취와 복습 루틴

오늘 할 수 있게 된 것

아래 다섯 가지를 코드를 보지 않고 할 수 있다면 오늘 수업은 소화한 것입니다.

#	할 수 있게 된 것	확인 방법
1	한글이 안 깨지는 그래프를 그린다	빈 노트북에서 rcParams 두 줄을 외워 쓸 수 있나
2	네 가지 그래프를 목적에 맞게 고른다	"분포를 보고 싶다" → 히스토그램이 바로 떠오르나
3	제목·축·범례가 갖춰진 그래프를 만든다	남이 봐도 설명 없이 읽히나
4	그림에서 이상 신호를 찾아낸다	865.6도가 왜 눈에 띄는지 설명할 수 있나
5	여러 그래프를 한 장으로 묶는다	subplot(2,2,1)의 세 숫자 의미를 아나

복습 루틴 — 3일 계획

시점	할 일	시간
오늘 저녁	17_열처리.csv로 네 가지 그래프를 코드 안 보고 다시 그려 보기	30분
내일	부록 A-5 표준 순서를 손으로 써 보고, 빠뜨린 줄이 있는지 대조	15분
주말	17_열처리_공정.csv로 2×2 리포트를 처음부터 끝까지 혼자 완성	40분

주의

시각화는 읽기만 해서는 절대 늘지 않습니다. 손으로 그려 보면 반드시 어딘가 막히는데, 그 막히는 지점이 실제로 모르는 부분입니다. 코드를 눈으로 읽을 때는 다 아는 것 같지만, 빈 셀에서 시작하면 plt.figure()를 어디에 놓아야 하는지부터 헛갈립니다. 한 번 막혀 봐야 외워집니다.

데이터 분석에서 그래프는 결과물이 아니라 사고 도구입니다. 오늘 실습 1·2에서 발견한 이상한 점이 실습 5의 산점도로 풀렸듯이, 그림이 다음 질문을 만들어 냅니다. 앞으로 어떤 데이터를 받든 먼저 그려 보는 습관을 들이십시오. 표만 보고 있으면 절대 보이지 않던 것이 그림에서는 3초 만에 보입니다.

수고하셨습니다.